Labo 15 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — `hostPath` pour une base

Réponse. Cela « marche » parce que k3s n'a qu'un nœud : `/data/pg` est toujours le même répertoire, sur la même machine. Sur trois nœuds, le Pod recréé après une panne atterrit sur un autre nœud, trouve un `/data/pg` **vide**, et PostgreSQL initialise une base neuve : les données sont « perdues » (en fait restées sur le nœud mort). La chaîne PVC → PV → StorageClass apporte un volume **indépendant du nœud** (disque réseau attaché là où le Pod tourne), un cycle de vie géré par le cluster, et une demande portable : la PVC ne dit pas *où*, seulement *combien* et *comment*.

Pourquoi. `hostPath` lie le Pod à une machine ; une PVC lie le Pod à un volume, que la plateforme sait déplacer.

Nuance. Avec `local-path`, le PV reçoit une **affinité de nœud** : le Pod ne sera jamais planifié ailleurs — même limite que `hostPath`, mais déclarée et visible (`Pending` avec `FailedScheduling` au lieu d'une base vide).

Exemple.

```
kubectl get pv -o
jsonpath='{.items[0].spec.nodeAffinity.required.nodeSelectorTerms[0].matchExpressions[0].values
[0]}' ; echo   # wsl-ubuntu
```

Question 2 — `Pending`, et c'est normal

Réponse. Ce n'est pas une erreur : la StorageClass `local-path` a `volumeBindingMode: WaitForFirstConsumer`. Le volume n'est créé — et la PVC liée — qu'au moment où un **Pod** qui référence la PVC est planifié, pour pouvoir créer le volume sur le nœud choisi. Dès qu'un Pod l'utilise, la PVC passe `Bound` en quelques secondes.

Pourquoi. Créer le volume avant de savoir où tournera le Pod obligerait le scheduler à suivre le volume ; l'inverse est plus sain.

Nuance. Avec `volumeBindingMode: Immediate` (certaines classes réseau), la PVC est liée tout de suite. `kubectl describe pvc` montre `waiting for first consumer to be created before binding` : la phrase à reconnaître.

Exemple.

```
kubectl describe pvc data | grep -i 'waiting for first consumer'
```

Question 3 — Le sort des données

Réponse. (a) Suppression du Pod : **rien** n'arrive aux données ; la PVC reste `Bound`, un nouveau Pod la remonte. (b) Suppression de la PVC : le PV, en politique `Delete`, est **supprimé avec ses données**. (c) Suppression du namespace : toutes les PVC du namespace sont supprimées, donc (b) pour chacune. (d) `replicas: 0` sur un StatefulSet : les Pods disparaissent, les PVC `data-db-0` **restent** ; revenir à `replicas: 1` les rattache.

Pourquoi. La PVC est l'objet qui « tient » le volume ; tant qu'elle existe, les données existent ; sa suppression déclenche la politique de récupération.

Nuance. `reclaimPolicy: Retain` garderait le PV (en état `Released`) après (b) et (c), à nettoyer ou à réattacher à la main — plus sûr, moins propre.

Exemple.

```
kubectl scale statefulset db --replicas=0 && kubectl get pvc     # data-db-0 toujours Bound
```

Question 4 — Deployment contre StatefulSet pour une base

Réponse. Deployment : Pod `db-7d9f8b6c5-x4k2p`, renommé à chaque recréation ; PVC `pgdata` écrite à la main, unique ; Service ClusterIP ordinaire ; à `replicas: 3`, les trois Pods tentent de monter la **même** PVC `ReadWriteOnce` — deux restent `Pending` (`Multi-Attach error`) ou, sur un seul nœud, trois PostgreSQL se disputent le même répertoire. StatefulSet : Pod `db-0`, nom conservé ; PVC `data-db-0` générée par `volumeClaimTemplates` ; Service headless ; à `replicas: 3`, trois Pods `db-0`, `db-1`, `db-2` avec **trois** PVC distinctes, démarrés dans l'ordre — trois bases indépendantes.

Pourquoi. Le Deployment considère ses Pods interchangeables ; le StatefulSet leur donne une identité et un stockage propres.

Nuance. Pour une seule instance qui ne sera jamais mise à l'échelle, un Deployment avec une PVC et `strategy: Recreate` (pas de rolling update, pour ne jamais avoir deux Pods sur le disque) est acceptable ; le StatefulSet reste plus explicite.

Exemple.

```
kubectl get pods,pvc -l app=db    # db-0 / data-db-0
```

Question 5 — Le Service headless

Réponse. Un Service headless a `clusterIP: None` : pas d'IP virtuelle, pas de répartition par kube-proxy. Le DNS `db.lab15.svc.cluster.local` renvoie directement les IP des Pods, et chaque Pod du StatefulSet reçoit en plus son propre nom : `db-0.db.lab15.svc.cluster.local`. Le Service `api` du labo 13 cachait les Pods derrière une IP stable ; le headless les **expose** individuellement. Un StatefulSet l'exige (`serviceName`) parce que c'est lui qui crée ces noms par Pod — indispensables pour qu'une réplique sache joindre `db-0` précisément.

Pourquoi. Une base n'est pas un ensemble de Pods interchangeables : le client doit parfois choisir lequel (primaire, réplica).

Nuance. On ajoute souvent un second Service **classique** `db-rw` avec un selector qui ne vise que le primaire (un label posé par l'opérateur) : les clients écrivent sur un nom stable, l'opérateur déplace le label à la bascule.

Exemple.

```
kubectl exec client -- getent hosts db db-0.db     # même IP, deux noms
```

Question 6 — `lost+found`

Réponse. Le volume est formaté en `ext4` par le pilote, et `ext4` crée un répertoire `lost+found` à sa racine : pour `initdb`, le répertoire « n'est pas vide ». C'est le kubelet (via le pilote de stockage) qui l'a mis là. La variable `PGDATA=/var/lib/postgresql/data/pgdata` fait initialiser la base dans un **sous-répertoire** vide du volume, ce que l'image officielle recommande.

Pourquoi. `initdb` refuse tout répertoire non vide pour ne jamais écraser une base existante par erreur.

Nuance. Avec `local-path`, le volume est un simple répertoire sans `lost+found` : le problème n'apparaît que sur un vrai disque. Mettre `PGDATA` dans un sous-répertoire dès le départ évite la surprise à la migration.

Exemple.

```
kubectl exec db-0 -- ls /var/lib/postgresql/data     # pgdata (et lost+found sur un disque formaté)
```

Question 7 — `local-path` sur deux nœuds

Réponse. Le volume est créé **sur le nœud où le premier Pod est planifié** (d'où `WaitForFirstConsumer`), dans `/var/lib/rancher/k3s/storage/`, et le PV reçoit une `nodeAffinity` vers ce nœud. Si ce nœud tombe, le Pod recréé ne peut être planifié que là : il reste `Pending` avec `FailedScheduling … node(s) had volume node affinity conflict` jusqu'au retour du nœud. Sur EKS, la StorageClass `gp3` (pilote EBS CSI) crée un disque réseau attachable à n'importe quel nœud de la zone ; la PVC **ne change pas** (sauf `storageClassName` si la classe par défaut diffère).

Pourquoi. `local-path` est fait pour un poste ou de l'edge ; il expose honnêtement sa limite dans le PV.

Nuance. Même EBS a une limite : le disque est lié à une **zone de disponibilité** ; un cluster multi-zones utilise `WaitForFirstConsumer` pour la même raison.

Exemple.

```
kubectl get pv -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeAffinity.required.nodeSelectorTerms[0].matchExpressions[0].values[0]
```

Question 8 — Trois bases, pas une base répliquée

Réponse. Ils ont déployé **trois PostgreSQL indépendants** : `db-0`, `db-1`, `db-2`, chacun avec sa PVC et sa base vide. Kubernetes ne sait rien de la réplication PostgreSQL (configuration du primaire, *streaming replication*, promotion d'un réplica) : il ne gère que des Pods et des volumes. Il faut un **opérateur** (CloudNativePG, Zalando Postgres Operator, Crunchy) : il crée le primaire, configure les réplicas, pose des labels pour les Services lecture/écriture, surveille, bascule, et sauvegarde.

Pourquoi. La haute disponibilité d'une base est un problème **applicatif** ; le StatefulSet n'apporte que les briques (noms, volumes, ordre).

Nuance. Pour des systèmes conçus pour le cluster (Cassandra, etcd, Elasticsearch), le StatefulSet suffit parfois parce que chaque nœud découvre les autres par les noms `db-N` ; PostgreSQL n'est

pas de ceux-là.

Exemple.

```
kubectl exec db-1 -- psql -U postgres -c '\dt'     # Did not find any relations : base vide
```

Question 9 — Supprimer le StatefulSet

Réponse. `get pvc` montre `data-db-0` toujours `Bound` : la suppression du StatefulSet ne touche pas aux PVC. Après `apply`, `db-0` est recréé, retrouve `data-db-0` par son nom, et le `SELECT` renvoie `2`. C'est voulu : un StatefulSet supprimé par erreur (ou remplacé par une nouvelle version) ne doit jamais emporter les données. Pour les faire disparaître : `kubectl delete pvc data-db-0` (politique `Delete`), ou supprimer le namespace.

Pourquoi. La PVC appartient au namespace, pas au StatefulSet ; seul `persistentVolumeClaimRetentionPolicy` (configurable) change ce comportement.

Nuance. Les PVC orphelines coûtent du stockage et s'oublient : `kubectl get pvc` régulier sur les namespaces partagés.

Exemple.

```
kubectl delete statefulset db && kubectl get pvc     # data-db-0 Bound
```

Question 10 — Les modes d'accès

Réponse. `ReadWriteOnce` : un disque bloc (EBS, Azure Disk, `local-path`) monté en lecture-écriture par **un seul nœud** — une base de données. `ReadOnlyMany` : un volume en lecture seule par plusieurs nœuds — des données de référence, un modèle. `ReadWriteMany` : un système de fichiers partagé (NFS, EFS, Azure Files, CephFS) — des uploads lus par plusieurs réplicas d'un front. Un disque bloc ne peut pas être `ReadWriteMany` parce que deux noyaux qui écrivent sur le même système de fichiers `ext4` sans coordination le corrompent. Deux Pods sur deux nœuds avec la même PVC `ReadWriteOnce` : le second reste `ContainerCreating` avec `Multi-Attach error for volume`.

Pourquoi. Le mode décrit ce que le stockage sous-jacent sait garantir, pas un souhait.

Nuance. `ReadWriteOncePod` (récent) restreint même à un seul Pod sur le nœud : utile pour une base.

Exemple.

```
kubectl get pvc -o custom-columns=NAME:.metadata.name,MODES:.spec.accessModes
```

Question 11 — `rsync` du répertoire du nœud

Réponse. Même problème qu'au labo 06 : une copie de fichiers pendant que PostgreSQL écrit donne un état incohérent, parfois irrécupérable — et ici en plus, le chemin sur le nœud est un détail d'implémentation de `local-path`. Dans un cluster : (1) la **sauvegarde logique**, `kubectl exec db-0 -- pg_dump -U postgres > dump.sql`, souvent dans un `CronJob` qui pousse vers un stockage objet ; (2) le **snapshot de volume** (`VolumeSnapshot`) si le pilote CSI le permet, cohérent au niveau bloc, à combiner avec `pg_start_backup` ou un arrêt. Un opérateur automatise : sauvegardes de base + archivage continu des WAL vers S3, et restauration à un instant donné.

Pourquoi. Le volume appartient à la plateforme ; la sauvegarde doit passer par l'application ou par l'API de stockage, pas par le nœud.

Nuance. Tester la restauration dans un namespace jetable fait partie de la procédure, comme au labo 06.

Exemple.

```
kubectl exec db-0 -- pg_dump -U postgres | gzip > db-$(date +%F).sql.gz
```

Question 12 — Rolling update d'un StatefulSet

Réponse. Un StatefulSet met à jour ses Pods **un à la fois**, dans l'ordre **décroissant** (`db-2`, puis `db-1`, puis `db-0`), en attendant que chacun soit `Ready` avant le suivant ; il n'a pas de `maxSurge` (jamais un Pod en plus : il n'y aurait pas de PVC pour lui) et détruit le Pod avant de le recréer. Un Deployment crée d'abord le nouveau Pod puis retire l'ancien, plusieurs en parallèle. Pour une base, c'est nécessaire : deux instances sur le même volume sont interdites, et l'ordre inverse met à jour les réplicas avant le primaire.

Pourquoi. L'identité et le volume par Pod imposent qu'un nom ne soit porté que par un Pod à la fois.

Nuance. `updateStrategy.rollingUpdate.partition` permet de ne mettre à jour que les Pods d'indice ≥ N : un *canary* sur `db-2` avant le reste.

Exemple.

```
kubectl get statefulset db -o jsonpath='{.spec.updateStrategy}' ; echo
```

Question 13 — Base managée ou base dans le cluster

Réponse. Critères : (1) **compétence et charge d'exploitation** — l'équipe veut-elle gérer sauvegardes, mises à jour, réplication, ou payer pour que le fournisseur le fasse ; (2) **exigence de disponibilité** — un service managé offre la bascule multi-zones en une case à cocher ; (3) **contraintes** — données qui doivent rester sur site, coût, latence, environnements de test nombreux (où un StatefulSet jetable est imbattable). Ce que l'application voit : base managée → un Service `ExternalName` (ou juste une URL en ConfigMap) et un Secret avec les identifiants ; base dans le cluster → un Service vers `db-0`/le primaire de l'opérateur, et le même Secret. Les manifestes de l'API sont **identiques** : `SPRING_DATASOURCE_URL` et un Secret, c'est tout.

Pourquoi. L'API ne connaît qu'un nom et un mot de passe ; l'emplacement de la base est une décision de plateforme.

Nuance. Beaucoup d'équipes font les deux : opérateur dans le cluster pour dev et recette, service managé en production.

Exemple.

```
kind: Service
metadata: { name: db }
spec: { type: ExternalName, externalName: app-prod.abc123.eu-west-1.rds.amazonaws.com }
```